PSAI_119

HAMSTER

- **Team Name:** Hamster
- **Payment Address:** 0xa5dEFB39eDF3B678D3C4F264EAA909E3f490d2D0(USDT)
- **Level**: 2

PROJECT OVERVIEW :PAGE_FACING_UP:

OVERVIEW

Hamster is a decentralized computing network based on the underlying chain. It aims to provide users with cost-effective computing servers. It is a decentralized platform for leasing computing resources that can be performed to rent idle computing resources

Currently there are many cloud computing service vendors providing cloud computing services, but currently with the expansion of services, the price of cloud computing services is gradually stabilized and high, and most of the cloud computing services are purchased on an annual basis, which causes great waste for current users who may use them for a short period of time, while the world's hardware capacity is gradually increasing today, and people have more and more idle computer products on hand. The goal of our product is to utilize the current idle computing resources in people's hands in a decentralized way, so that users who need it can use computing services in a cost-effective way.

Our team aims to build a complete platform for leasing idle computing resources. The decentralized computing resource leasing function is realized by using substrate framework pallet, which can be leased and traded on the chain, and the link and use of computing resources can be done through the p2p network protocol under the chain after signing the lease agreement on the chain.

PROJECT DETAILS

Hamster is composed of Hamster nodes, Hamster resource providers, Hamster clients, Hamster gateways, and Hamster front-end pages.



- Hamster Node: is a custom node built on Substrate 3.0.
  - pallet_provider: performs resource provider registration and provider resource information storage, and provides a computational marketplace
  - pallet_resource-order: performs resource lease order functions and lease agreement execution
  - pallet_gateway: The gateway mainly has the following functions: gateway registration, gateway heartbeat detection, gateway status reset, gateway drop penalty, and receive rewards. The main role is to add the gateway as an important player in the shared computing platform
- Hamster Gateway: p2p gateway with public IP, used to link information between resource provider and resource user, built with libp2p component, is the cornerstone of the leased resource availability, can register itself to Hamster Node. That includes the **Register**, **Receive rewards** and **Configuration** modules.
  - Register: Register Gateway information to Hamster Node.
  - Receive rewards: Revenue for those who provide gateway resources.
  - Configuration: Basic configuration of p2p gateway information.
- Hamster Provider: can provide compute resources and register them with Hamster Node. Compute resources are provided using both vm virtual machine technology and docker technology. Currently vm virtual machine technology is used to better protect user privacy. That includes the **Initialize configuration**, **Resource details**, **Account information** and **Configuration information** modules.
  - Initialize configuration: Initialize configuration, including p2p seed node configuration, p2p port configuration.
  - Resource details: Available spare resource specifications (cpu, memory, etc.), and price.
  - Account information: Provide the import of the substrate account, provide the pledge of the deposit before the service, etc.
  - Configuration information: resources that have not reached a transaction are offline at any time, resource specifications, price configuration, etc.
- Hamster Client: After purchasing compute resources in the front-end marketplace, users can view them through the client and link to them. That includes the **Market**, **My order** and **My resource** modules.
  - Market: A trading market where computing power providers submit idle computing power to the market and configure prices. The client can choose the configuration and price resources to be purchased to form a transaction contract.
  - My Orders: List and details of all resource orders I have purchased.
  - My resources: The list of resources corresponding to the current valid order, the client app can establish a connection with the remote resources through the list of resources.
- Hamster Front End: Hamster Dapp, which allows users to purchase compute resources that have already been provided, and pay a certain Token to purchase a certain amount of time to use the compute resources. We can pay by the hour for more flexible use.

Project Flow



Anti-malicious attacks

- How do you ensure the system isn't exploited and users actually fulfill the agreement?

When the agreement is reached, the provider is required to complete the creation of the virtual machine, notify the blockchain of the completion of the order agreement, and request a heartbeat per epoch, which includes the number of cpu cores and memory size, for the blockchain to verify that it is working and providing valid resources. The potential threat is a nefarious provider forging heartbeats to trick the network into providing a valid service that actually provides less than the promised resources or even none at all. Since we have thought long and hard about this, and we do not currently have a cryptographic or better design to accomplish such a solution, we have adopted the idea of an arbitration model to prevent malicious attacks.



1. when the user finds a problem with the resource and the information configured at the time of purchase does not match, submits a validation request and requests arbitration. in order to ensure that there is no malicious validation, the submission of the request will have a certain cost pledge.
2. the arbitration group is subject to the validation request, currently designed to arbitrate the group for the current chain of validators. because he is the block validator, will promote the role of the current ecology. It is a trusted team
3. the arbitration group to link the computing resources, after entering the link for linux least privilege account, to query the virtual machine specifications
4. arbitration break to verify and report the vote
5. according to the voting results for incentives and penalties, to achieve the purpose of malicious evil.

We will follow up with cryptography or a better algorithmic model to replace the arbitration model. This will be a major topic of our subsequent research.

- What are other potential attack vectors and how are you going to address them?
    - Virtual machine p2p connection security:

      The connection port of the virtual machine is exposed to the p2p network and is vulnerable to security attacks from the p2p network. When provier creates a virtual machine, it will use the public key of the purchasing user and inject it into the virtual machine. The password will be changed to a random password to ensure that no one can access it, and disable remote login by password. This protects the login security of the virtual machine.

    - Host network security:

      Mainly facing Escape Exploit . The ‘Escape Exploit’ problem is a persistent network security attack and needs to be solved by updating the corresponding software in time. We will iterate on the provider virtualization technology in the future.

Project Technology Stacks

- Rust
- Substrate
- Node.js
- Golang
- Wails
- Libp2p
- Kvm
- Docker
- Vue.js
- Polkadot.js

Proof of Concept

The current project is in the primary stage of validation of the entire business process, the Gateway module has not yet been completed, this is the goal of our subsequent planning.

Project page

- Resource Registration

  

- Resource Purchase

  

- Resource Usage

  

ECOSYSTEM FIT

Our project is a distributed shared computing platform. Through the shared computing platform, our target customers are user groups with idle resource machines and some users who want to use cost-effective resources, they can be developers, designers, etc., and can deploy their inspirations that they want to share to our shared computing network to share out. The follow-up vision is to build an ecosystem of shared services on the shared platform network

TEAM :BUSTS_IN_SILHOUETTE:

TEAM MEMBERS

- Teng Liang
- Haijiang Mo
- Jianguo Sun
- Zhiwei Li

CONTACT

- **Contact Name:** Teng Liang
- **Contact Email:** ltyuanmu@gmail.com
- **Website:** Hamster is currently maintained on Github, no website has been created yet

LEGAL STRUCTURE

TEAM'S EXPERIENCE

Teng Liang: Currently a team project leader with 8 years of software development experience, designed in the areas of Cloud Computing, Cloud Native, DevOps. and familiar with Java/Go/Rust languages and Solidity smart contract development.

Haijiang Mo: CTO of the team, with 10 years of software development experience. He is familiar with Go/Rust/Java/Python/Javascript/Typescript and has worked in cloud computing, cloud native, DevOps.

Jianguo Sun : Full-stack engineer with three years of development experience. And familiar with Go/Rust/Java/Python/Javascript/Typescript, front-end and client-side development.

Zhiwei Li: Graduate student in Computer Science, Substrate pallet developer.

TEAM CODE REPOS

- https://github.com/hamster-shared
- https://github.com/hamster-shared/frontend
- https://github.com/hamster-shared/hamster
- https://github.com/hamster-shared/hamster-client
- https://github.com/hamster-shared/hamster-provider
- https://github.com/ltmuyuan
- https://github.com/mohaijiang
- https://github.com/jian-guo-s
- https://github.com/LwwL-123

DEVELOPMENT STATUS :OPEN_BOOK:

We have completed a simple version of the project for minimal process execution, and will subsequently update the usage documentation so you can install and experience using it.

The following is a list of the features that have been implemented:

- Hamster Node

  Computer resource register

  Update computer resource unit-price

  Create resource order

  Provider finish order

  Provider heartbeat

  Provider staking amount

  Provider withdraw amount

  Withdraw rental amount

  Withdraw fault execution

  Client cancel order

- Hamster Provider

  Computer create

  Computer destruction

  P2p link listen

- Hamster Client

  List of paid computers

  Computer link usage

- Hamster Front End

  Computing market

  Provider shared computing

  Provider lease agreement list

  List of computers purchased by the customer

  Customer order list

- Hamster Gateway

  Private IPFs network

DEVELOPMENT ROADMAP :NUT_AND_BOLT:

OVERVIEW

- **Total Estimated Duration:** 2 months
- **Full-Time Equivalent (FTE):** 4 FTE
- **Total Costs:** $25,600

MILESTONE 1 EXAMPLE — IMPLEMENT HAMSTER CLIENT AND PROVIDER MODULES

- **Estimated duration:** 1 month
- **FTE:** 4
- **Costs:** $12,800

Number	Deliverable	Specification
0a.	License	Apache 2.0
0b.	Documentation	We will provide online documentation of the code and a basic tutorial that includes 1. Hamster Client installation tutorial 2. Hamster Client usage tutorial 3. Hamster Provider installation tutorial 4. Hamster Provider usage tutorial 5. Hamster Chain installation tutorial 6. Hamster Chain usage tutorial
0c.	Testing Guide	Core functions will be fully covered by unit tests to ensure functionality and robustness. In the guide, we will describe how to run these tests.
1.	Hamster Client	We will create a desktop client that will have **Market**,**My order** and **My resource** module. We will provide a desktop (windows, macos,linux) app based on wails to achieve 1: Market inquiry and management of purchased resources, 2: The point-to-point communication between the client and the computing resource provider. 3: Query historical orders 4: Partial fee refund for defaulted orders Tech stack: go+wails+go-libp2p+vue.js+polkadot.js
2.	Hamster Provider	We will create a a resource provider server that will have **Initialize configuration**,**Resource details**,**Account information** and **Configuration information** module. The resource provider will use the idle resources of the machine, register in the market, and declare that it can provide rental. When the provided virtual machine reaches a transaction in the market, the resource provider provides the virtual machine with the corresponding quota for the remote purchaser to use according to the agreement. The resource provider will establish a p2p connection with the used client for remote management, such as ssh, rdesktop, etc. Our initial idea is to use the libvirt scheme to implement the management of virtual machines. (Windows uses Hyper-V) We will support these operating systems: Windows 10, Ubuntu 18+, CentOS 7+ Tech stack: go+go-libp2p+libvirt+go-substrate-rpc-client+gin
		Web app is a set of web management tools of Provider. Through this set of management tools, users can share or stop sharing idle resources for their own use, set prices for their own idle resources, adjust the specifications of idle resources provided, and modify

3.	Hamster Provider: web app	the provisioning services of idle resources. period, etc. And when a transaction is terminated normally, the desired compensation can be obtained through the contract. (Providing idle resource services requires a certain pledge. After the transaction is breached, part of the pledge deposit will be deducted. If the pledge deposit is too low, other people will not be able to see this idle shared resource in the market) Tech stack: vue.js+polkadot.js+Node.js
4.	Hamster chain	pallet_provider, pallet_resource-order modifications and optimizations due to the need to optimize and adapt already developed pallets when adding features for customers. As code delivery of the underlying framework, there are two integrated pallets
5.	Whitepaper	Preparation of project white papers

MILESTONE 2 EXAMPLE — IMPLEMENT HAMSTER GATEWAY MODULES

- **Estimated duration:** 1 month
- **FTE:** 4
- **Costs:** $12,800

Number	Deliverable	Specification
0a.	License	Apache 2.0
0b.	Documentation	We will provide online documentation of the code and a basic tutorial that includes 1. Hamster Gateway installation tutorial 2. Hamster Gateway usage tutorial
0c.	Testing Guide	Core functions will be fully covered by unit tests to ensure functionality and robustness. In the guide, we will describe how to run these tests.
0d.	Article	We will write an article or tutorial that explains the work done as part of the grant.
1.	Hamster Gateway	We will create a desktop client that will have **Register**, **Receive rewards** and **Configuration** module. Tech stack: go+go-libp2p+libvirt+go-substrate-rpc-client+gin
2.	Hamster Node: pallet_gateway	We will create a Substrate module that will have **Register gateway**,**Receive rewards** etc function.
3.	Hamster Gateway: web app	We will create a web app integrated in the Hamster Gateway, to let users easily interact with our Hamster Gateway function module. Tech stack: vue.js+polkadot.js+Node.js
4.	Hamster chain	Add a gateway module in the Hamster chain, to enable users to share gateway resources with public IP , receive rewards, etc.

FUTURE PLANS

Our current goal is to provide distributed shared computing power. After this milestone is completed, we will be able to build our own ecosystem based on resources. That is, when everyone is willing to add their free resources to the hamster network, it can be assumed that there are near-infinite computing resources in the network, and more quality services can be built and constructed in the hamster network. For example, service providers no longer need to rely on the support of a single cloud vendor, but only need to use the computing resources in the hamster network, and the system automatically schedules the resource arithmetic needed for the services to build their own services in the form of edge computing. In addition to this, we can also provide some toolkits so that these computing resources can participate in other services with one click, such as becoming a chain node of Thegraph, so that they can participate in other networks, and later on, we can also use incentives to encourage people to develop toolkits for different blockchain ecosystems. When more and more people and more service providers participate in the hamster network, maybe it can become a truly decentralized metaverse service cornerstone.

ADDITIONAL INFORMATION :HEAVY_PLUS_SIGN:

So far, we are self-funded development. However, for our long-term vision, we will try to get support from investment institutions.